DA5402 — MLOps

Project Report

Melanoma Detection MLOps Pipeline

End-to-End MLOps System for Low-Resolution Skin Lesion Classification

Submitted by

Dikshank Sihag

Roll Number: DA25M009

April 2026

1. Abstract

This report presents the design, implementation, and evaluation of a production-grade MLOps pipeline for melanoma detection from low-resolution skin lesion images. The system is built entirely on CPU-only, on-premise infrastructure — no cloud services are used — and demonstrates how best-practice MLOps principles can be applied in resource-constrained clinical screening environments.

The pipeline covers the complete machine learning lifecycle: automated data ingestion and preprocessing via Apache Airflow, multi-model training and experiment tracking via MLflow, REST API serving via FastAPI, near-real-time monitoring via Prometheus and Grafana, and automated model retraining triggered by performance drift or user feedback. Three model architectures — MobileNetV3-Small, EfficientNet-B0, and a custom SimpleCNN — were trained and compared using a champion/challenger auto-promotion framework with recall as the primary metric and F1 as a tiebreaker. All 68 automated tests pass at 100%, with average prediction latency of approximately 150ms and zero production errors during evaluation.

2. Introduction

2.1 Problem Statement

Melanoma is one of the most dangerous forms of skin cancer. Early detection significantly improves patient outcomes, yet access to specialist dermatologists remains limited in many healthcare settings, particularly in rural or low-resource environments. Patients in these contexts typically have access only to basic mobile devices capable of capturing low-resolution images (32×32 pixels).

This project builds an end-to-end MLOps system — named MelanoCheck — that assists non-specialist healthcare workers in deciding whether to escalate a patient for professional dermatological review. The system is designed for on-premise deployment with no cloud dependency, using only CPU resources.

2.2 Objectives

- Design and implement a reproducible MLOps pipeline from raw data ingestion to model serving.
- Train and compare three CNN architectures on low-resolution skin lesion images.
- Implement automated champion/challenger model promotion based on recall and F1.
- Build a production-grade REST API with drift detection and feedback collection.
- Monitor system health and model performance in near-real-time using Prometheus and Grafana.
- Implement automated retraining triggered by monitoring alerts.
- Achieve 100% pass rate on 68 automated tests.

2.3 Design Philosophy

The system is built on the functional programming paradigm. Pipeline scripts, utility modules, and API handlers are implemented as pure functions with clearly defined inputs and outputs. Side effects (file I/O, model loading, logging) are isolated to dedicated modules. The only exceptions are PyTorch Dataset and nn.Module subclasses, which are required by the framework.

3. System Architecture

3.1 High-Level Architecture

The system is composed of six horizontal layers, each implemented as one or more Docker containers communicating over a shared mlops_network:

Layer	Components	Technology
Data Layer	Raw image storage + DVC tracking	DVC + Git LFS
Pipeline Layer	Data validation, splitting, preprocessing, baseline stats	Apache Airflow 2.8.1
Training Layer	3-model training, experiment tracking	PyTorch 2.x + MLflow
Serving Layer	REST API + static frontend	FastAPI + nginx
Monitoring Layer	Metrics collection + dashboards + alerts	Prometheus + Grafana
Retraining Layer	Auto-retraining DAG + feedback loop	Apache Airflow

3.2 Data Flow

Raw 32×32 pixel skin lesion images are stored locally and versioned with DVC. The Airflow data pipeline DAG validates images, performs stratified train/val/test splits, upscales images to 64×64, and computes pixel histograms for drift detection baselines. Processed data is stored on the local filesystem and mounted into training and serving containers.

Training runs produce MLflow experiment logs for all three model architectures. A champion/challenger promotion algorithm selects the best model and registers it to the MLflow Production stage. The FastAPI backend loads the Production model at startup and serves predictions via REST API. Every prediction is saved for potential retraining, and users can submit ground-truth feedback to update the real-world performance metrics.

Prometheus scrapes the /metrics endpoint every 15 seconds and feeds Grafana's 13-panel dashboard. When misclassification rate exceeds 10% or drift exceeds 20%, the Airflow retraining DAG is triggered automatically every 30 minutes.

3.3 Docker Compose Services

Service	Image	Port	Role
melanoma_frontend	nginx:alpine	80	Static UI (Screening + Dashboard tabs)
melanoma_backend	Custom Python	8000	FastAPI REST API + inference
melanoma_mlflow	MLflow	5000	Experiment tracking + model registry
melanoma_prometheus	prom/prometheus	9090	Metrics collection
melanoma_grafana	grafana/grafana	3000	NRT dashboards + alerting

3.4 Key Design Decisions

Decision	Rationale
SequentialExecutor + SQLite for Airflow	No additional containers needed; suitable for single-machine deployment
CPU-only inference	On-premise constraint (Intel UHD 620); no GPU available
32×32 → 64×64 upscaling	Proves resolution is the bottleneck; models are ready for higher-resolution data in future without config changes
Recall as primary metric	Missing cancer (false negative) is clinically more dangerous than a false alarm
F1 as tiebreaker	Prevents degenerate models that flag everything as malignant from being promoted
Local DVC remote	On-premise constraint — no cloud storage used
Functional programming paradigm	Testable, composable, side-effect-isolated pipeline stages
Colorblind-friendly UI	Amber/blue instead of red/green; accessible to deuteranopic users
Configurable API_BASE	window.API_BASE ensures loose coupling between frontend and backend

4. Data Pipeline

4.1 Dataset

The dataset consists of 801 skin lesion images in two classes: malignant and benign. Raw images are stored at 32×32 pixel resolution, simulating the constraints of basic mobile camera hardware. All images are versioned with DVC using a local remote, with the .dvc pointer file committed to Git for reproducibility.

4.2 Airflow DAG — melanoma_data_pipeline

The data pipeline is implemented as a 4-task Airflow DAG using the SequentialExecutor:

- validate_images: Checks for corrupt or zero-byte files, enforces minimum class image counts, and returns a validation summary dictionary.
- split_data: Detects flat vs pre-split dataset structure, performs stratified 70/15/15 train/val/test splits, and produces CSV manifests with filepath and label columns.
- preprocess_images: Resizes all images from 32×32 to 64×64 pixels using bilinear interpolation, converts to RGB, and saves to versioned output folders.
- compute_baseline_stats: Computes per-channel pixel intensity histograms (10 bins) over the training set. These histograms form the baseline reference for drift detection at serving time.

4.3 Outputs

Output	Description
data/processed/v1/train/	164 training images at 64×64 RGB
data/processed/v1/val/	36 validation images
data/processed/v1/test/	600 test images
data/baseline_stats/v1_stats.json	Pixel histogram baseline for drift detection
CSV manifests	filepath + label columns for each split

5. Model Training

5.1 Model Architectures

Three model architectures are trained and compared in each experiment run:

Model	Type	Parameters	Trainable Params	Notes
MobileNetV3-Small	Pretrained (ImageNet)	1.5M	592K	Lightweight, CPU-friendly
EfficientNet-B0	Pretrained (ImageNet)	4M	2K	Stronger feature extractor
SimpleCNN	From scratch	102K	102K	Proves resolution is bottleneck

All models share the same configuration: 64×64 RGB input tensors, CrossEntropyLoss with class weights [1.0, 2.0] to upweight the malignant class, and automatic threshold tuning on the validation set to maximise recall. Early stopping is applied with a patience of 3 epochs on validation recall.

5.2 Training Configuration

- Input: 64×64 RGB tensor, normalised with ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
- Loss: CrossEntropyLoss with class weights [1.0, 2.0] to account for the clinical cost asymmetry.
- Threshold: Auto-tuned on the validation set to maximise recall (not fixed at 0.5).
- Early stopping: patience=3 on validation recall to prevent overfitting.
- Experiments tracked in MLflow experiment baseline-v1, with per-epoch metrics, hyperparameters, and model artifacts logged.

5.3 Champion/Challenger Auto-Promotion

After every training run, a two-stage comparison determines whether the new model replaces the current Production model in the MLflow registry:

- Stage 1 — Recall (primary metric, patient safety): If new recall exceeds current recall by more than 0.02, the new model is promoted. If it is more than 0.02 below, the current model is retained.
- Stage 2 — F1 tiebreaker (when recall is within ±0.02): If the new model has a higher F1 score, it is promoted for better precision-recall balance. Otherwise, the current model is retained for stability.

This ensures that the FastAPI backend always serves the model that maximises patient safety while avoiding degenerate all-malignant classifiers.

5.4 Reproducibility

Every MLflow run is tagged with the Git commit hash, data version, model architecture name, and promotion outcome. Any experiment can be exactly reproduced using:

```
git checkout <commit_hash>  →  dvc checkout  →  mlflow run . -P
config=<config>
```

6. Serving Layer

6.1 FastAPI Backend

The backend is implemented in Python 3.10 using FastAPI 0.111.0 with Uvicorn. It exposes five REST endpoints, loads the Production model from the MLflow registry at startup, and runs entirely in a Docker container on port 8000.

Endpoint	Method	Purpose
/predict	POST	Classify uploaded image; save to pending_feedback; detect drift
/feedback	POST	Submit ground-truth label; update confusion matrix; trigger retraining flag
/health	GET	Liveness check — always returns 200 if service is running
/ready	GET	Readiness check — confirms model is loaded; returns model metadata
/metrics	GET	Prometheus exposition format — 15 custom metrics

6.2 Inference Pipeline

When a POST /predict request is received, the following steps are executed: the uploaded file is validated (JPEG or PNG only), a unique image_id is generated from an MD5 hash and timestamp, the image is saved to logs/pending_feedback/ for retraining, and the image is preprocessed (resized to 64×64, normalised) before being passed to the PyTorch model. Softmax probabilities are computed, the recall-optimised threshold is applied, and drift detection runs against the baseline histogram. All results are recorded to Prometheus and the PredictResponse is returned to the caller.

6.3 Drift Detection

On every prediction request, a 10-bin pixel intensity histogram is computed for the input image and compared against the baseline histogram stored in baseline_stats/v1_stats.json (produced by the Airflow data pipeline). The drift score is the mean absolute error between the two histograms. If the drift score exceeds 0.20, the system writes a retrain_needed.flag file and increments the retraining trigger counter in Prometheus.

6.4 Frontend

The frontend is a static HTML/CSS/JS application served by nginx on port 80. It provides two tabs: a Screening tab for image upload and result display, and a Pipeline Dashboard tab showing live system metrics, pipeline stage status, and links to Airflow, MLflow, Grafana, and Prometheus. The frontend communicates exclusively with the backend via a configurable REST API base URL (window.API_BASE), ensuring complete decoupling. The UI uses colorblind-friendly amber/blue colour coding instead of red/green.

7. Monitoring and Alerting

7.1 Prometheus Metrics

The FastAPI backend exposes 15 custom Prometheus metrics via the /metrics endpoint:

Metric	Type	Description
melanoma_request_total	Counter	Total requests by endpoint/method/status
melanoma_request_latency_seconds	Histogram	Request latency by endpoint
melanoma_prediction_total	Counter	Predictions by class label
melanoma_confidence_score	Histogram	Distribution of confidence scores
melanoma_drift_score	Gauge	Current pixel histogram MAE vs baseline
melanoma_drift_detected	Gauge	1 if drift > 20%, else 0
melanoma_real_world_recall	Gauge	Running recall from feedback (TP / (TP+FN))
melanoma_real_world_precision	Gauge	Running precision from feedback (TP / (TP+FP))
melanoma_misclassification_rate	Gauge	Current misclassification rate from feedback
melanoma_retraining_triggered	Counter	Number of retraining triggers
melanoma_model_loaded	Gauge	1 if Production model loaded, else 0

7.2 Grafana Dashboard

The Grafana dashboard (provisioned automatically from grafana/provisioning/) contains 13 panels displaying near-real-time metrics: request rate, prediction latency percentiles, prediction class distribution, drift score with threshold line, misclassification rate, real-world recall and precision, error rate, model loaded status, retraining trigger count, and feedback accumulation.

7.3 Alert Rules

Three alert rules are provisioned automatically:

- Error rate > 5% — Critical severity: triggers when the proportion of 5xx responses exceeds 5%.
- Drift detected — Warning severity: triggers when melanoma_drift_detected gauge equals 1.
- Misclassification rate > 10% — Warning severity: triggers when the real-world misclassification rate exceeds 10% (after at least 10 feedback submissions).

8. Automated Retraining

8.1 Feedback Loop

Medical professionals can submit ground-truth labels for previous predictions via the POST /feedback endpoint. Each submission moves the image from logs/pending_feedback/ to logs/feedback_data/<true_label>/, updates the confusion matrix (TP, FP, FN, TN), recomputes running recall and precision, and appends an audit entry to logs/feedback.jsonl. When the misclassification rate exceeds 10% (after at least 10 submissions), the retrain_needed.flag is written.

8.2 Airflow Retraining DAG

The melanoma_retraining_pipeline DAG runs automatically every 30 minutes and executes five tasks in sequence:

- check_trigger: Reads retrain_needed.flag. Stops the DAG early if the flag is not present.
- prepare_feedback_data: Merges confirmed feedback images into the training manifests.
- trigger_training: Retrains all three model architectures via train.py.
- evaluate_new_model: Applies the champion/challenger comparison against the current Production model in the MLflow registry.
- cleanup: Archives feedback images to logs/feedback_data_archive/<timestamp>/, removes the flag file.

8.3 Retraining Triggers

Trigger Condition	Threshold	Source
Misclassification rate	> 10% (after >= 10 feedback submissions)	POST /feedback → Prometheus → retrain_needed.flag
Drift score	> 20% histogram MAE	POST /predict → drift detection → retrain_needed.flag
Manual trigger	—	Airflow UI → melanoma_retraining_pipeline → Trigger DAG
Schedule	Every 30 minutes	Airflow cron schedule

9. Testing

9.1 Test Summary

Module	Tests	Passed	Failed	Pass Rate
POST /predict	15	15	0	100%
GET /health + /ready + /metrics	17	17	0	100%
POST /feedback	14	14	0	100%
Airflow pipeline scripts	22	22	0	100%
Total	68	68	0	100%

All 68 tests were executed on 26 April 2026 using pytest 9.0.3 on Python 3.14.3. Total execution time was 32.98 seconds. API tests use FastAPI's TestClient with mocked model state, so no running Docker services are required.

9.2 Test Coverage Areas

- POST /predict (15 tests): Valid JPEG/PNG uploads, response schema validation, confidence and probability range checks, edge cases (1×1px, 1000×1000px images), invalid file types, empty files, missing fields, and model-not-loaded 503 response.
- GET /health and /ready (11 tests): Liveness check with and without model, readiness check schema, model_loaded boolean type, status value validity, model version presence.
- GET /metrics (6 tests): HTTP 200, Prometheus content type, presence of key metric names, counter increment after prediction.
- POST /feedback (14 tests): All four feedback types (TP, TN, FP, FN), response schema, invalid label values (400), missing fields (422), real-world recall metric update, accumulation across multiple submissions.
- Airflow pipeline scripts (22 tests): Validation logic (corrupt files, missing folders, minimum counts), split structure detection, manifest creation and coverage, preprocessing output size and mode, baseline stats JSON structure and histogram correctness.

9.3 Acceptance Criteria Results

Criterion	Target	Actual	Status
All unit tests pass	100%	100% (68/68)	PASS
/predict latency	< 200ms	~150ms avg	PASS
Error rate	< 5%	0%	PASS
Docker services start	No errors	All 5 services up	PASS
Model loaded at startup	Yes	Version 13 (SimpleCNN)	PASS
Drift detection	Operational	Score=0.11, threshold=0.20	PASS
Retraining trigger	Operational	Triggers at >10% misclass	PASS
Grafana alert rules	3 rules	3 rules provisioned	PASS

10. Technology Stack

Component	Technology	Version
Data pipeline orchestration	Apache Airflow (SequentialExecutor + SQLite)	2.8.1
ML framework	PyTorch (CPU only)	2.x
Model architectures	MobileNetV3-Small, EfficientNet-B0, SimpleCNN	—
Experiment tracking	MLflow (local mlruns/)	Latest
Data versioning	DVC + Git LFS (local remote)	Latest
REST API serving	FastAPI + Uvicorn	0.111.0
Frontend serving	nginx:alpine	Latest
Metrics collection	Prometheus	Latest
Monitoring dashboards	Grafana (13 panels, 3 alerts)	Latest
Containerisation	Docker + Docker Compose (5 services)	Latest
CI pipeline	GitHub Actions + dvc.yaml DAG	—
Testing framework	pytest (68 tests)	9.0.3
Operating system	Windows 11 / Linux (Ubuntu 24)	—

11. Project Structure

The project is organised into logical top-level directories, each corresponding to a layer of the MLOps pipeline:

Directory / File	Contents
airflow/	Dockerfile, docker-compose-airflow.yml, data_pipeline_dag.py, retraining_dag.py, pipeline scripts, configs
training/	dataset.py, model.py, train.py, evaluate.py, mlflow_utils.py, per-model YAML configs
backend/	main.py (FastAPI), model_loader.py, predictor.py, monitoring.py, schemas.py, logger.py, Dockerfile
frontend/	index.html (Screening + Dashboard tabs), style.css, app.js (configurable API_BASE), Dockerfile
prometheus/	prometheus.yml (scrape config)
grafana/	provisioning/ — dashboards/melanoma.json (13 panels), datasources/, alerting/ (3 rules)
tests/	conftest.py, test_predict.py (15), test_health.py (17), test_feedback.py (14), test_pipeline.py (22)
scripts/	batch_predict.py — batch prediction and feedback script for demo

Directory / File	Contents
docs/	HLD.md, LLD.md, architecture.html, test_plan.md, test_report.md, user_manual.md
MLproject	MLflow Projects entry point for reproducible training runs
dvc.yaml	5-stage CI pipeline DAG
docker-compose.yml	5-service Docker Compose definition
.github/workflows/ci.yml	GitHub Actions CI — pytest + linting on every push

12. Conclusion

This project demonstrates a complete, production-grade MLOps pipeline for melanoma screening that operates entirely within on-premise, CPU-only constraints. The system successfully integrates data versioning, automated data processing, multi-model experimentation, champion/challenger model promotion, REST API serving, near-real-time monitoring, and automated retraining into a coherent and reproducible architecture.

Key achievements include: a 100% test pass rate across 68 automated tests; average prediction latency of approximately 150ms on CPU; zero production errors during evaluation; automated drift detection and retraining triggers; colorblind-accessible user interface; and full experiment reproducibility through Git, DVC, and MLflow integration.

The choice of recall as the primary metric, with F1 as a tiebreaker, reflects the clinical priority of minimising missed melanoma cases. The SimpleCNN from scratch achieving competitive performance against ImageNet-pretrained models (MobileNetV3-Small, EfficientNet-B0) confirms that image resolution is the primary bottleneck in this domain — a finding that motivates future data collection efforts to acquire higher-resolution images.

The system is designed for extensibility: upgrading to 128×128 or 256×256 images requires only a config change, and adding new model architectures requires only a new YAML config and training entry point. The monitoring infrastructure ensures that any performance degradation in production is detected and remediated automatically, with no manual intervention required.

References

- Apache Airflow Documentation — https://airflow.apache.org/docs/
- MLflow Documentation — https://mlflow.org/docs/latest/index.html
- FastAPI Documentation — https://fastapi.tiangolo.com/
- Prometheus Documentation — https://prometheus.io/docs/
- DVC Documentation — https://dvc.org/doc
- PyTorch Documentation — https://pytorch.org/docs/stable/index.html
- Howard, A. et al. (2019). Searching for MobileNetV3. ICCV 2019.
- Tan, M. & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.